Kisombwa Ranch - Complete Database Schema

Django Apps & Models Reference

📊 Database Overview

Total Tables: 12
Django Apps: 5 (core, livestock, iot, analytics, alerts)

🗂️ App 1: `apps/core/` - Foundation Models

Table: `users`

Django Model: `apps/core/models.py` → `User`
Extends: Django's AbstractUser

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
username	VARCHAR(150)	UNIQUE, NOT NULL	Login username
email	VARCHAR(254)	NOT NULL	Email address
password	VARCHAR(128)	NOT NULL	Hashed password
first_name	VARCHAR(150)	-	First name
last_name	VARCHAR(150)	-	Last name
role	VARCHAR(20)	NOT NULL, DEFAULT 'viewer'	admin/vet/worker/viewer
phone_number	VARCHAR(20)	-	Contact number
is_staff	BOOLEAN	DEFAULT false	Django admin access
is_active	BOOLEAN	DEFAULT true	Account status
is_superuser	BOOLEAN	DEFAULT false	Full permissions
date_joined	TIMESTAMP	NOT NULL	Account creation date
last_login	TIMESTAMP	-	Last login time

Indexes:

- PRIMARY KEY: `id`
- UNIQUE: `username`, `email`

Table: **`ranches`**

Django Model: `apps/core/models.py` → `Ranch`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
name	VARCHAR(200)	NOT NULL	Ranch name
location	VARCHAR(200)	NOT NULL	Physical location
size_hectares	DECIMAL(10,2)	NOT NULL	Ranch size
owner_id	UUID	FOREIGN KEY → users.id	Ranch owner
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `owner_id` → `users(id)` ON DELETE CASCADE

Table: **`animals`**

Django Model: `apps/core/models.py` → `Animal`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
tag_id	VARCHAR(50)	UNIQUE, NOT NULL, INDEXED	QR code identifier (e.g., BORAN001)
name	VARCHAR(100)	NOT NULL	Animal name
breed	VARCHAR(50)	NOT NULL	Breed (Boran, Ankole, etc.)
gender	VARCHAR(10)	NOT NULL	male/female
birth_date	DATE	NOT NULL	Date of birth
status	VARCHAR(20)	NOT NULL, DEFAULT 'active'	active/sick/quarantine/sold/deceased
ranch_id	UUID	FOREIGN KEY → ranches.id	Associated ranch
sire_id	UUID	FOREIGN KEY → animals.id, NULL	Father (self-reference)
dam_id	UUID	FOREIGN KEY → animals.id, NULL	Mother (self-reference)
photo	VARCHAR(100)	-	Path to animal photo
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- UNIQUE INDEX: `tag_id`
- FOREIGN KEY: `ranch_id` → `ranches(id)` ON DELETE CASCADE
- FOREIGN KEY: `sire_id` → `animals(id)` ON DELETE SET NULL
- FOREIGN KEY: `dam_id` → `animals(id)` ON DELETE SET NULL
- INDEX: `tag_id` (for fast QR lookups)

Computed Properties (not stored):

- `age_months` - Calculated from birth_date

🏥 App 2: `apps/livestock/` - Health & Lifecycle

Table: `health_records`

Django Model: `apps/livestock/models.py` → `HealthRecord`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
animal_id	UUID	FOREIGN KEY → animals.id	Associated animal
record_type	VARCHAR(50)	NOT NULL	checkup/treatment/injury/illness/surgery
date	DATE	NOT NULL	Record date
diagnosis	TEXT	-	Diagnosis details
treatment	TEXT	-	Treatment administered
medication	VARCHAR(200)	-	Medication name
dosage	VARCHAR(100)	-	Dosage information
veterinarian_id	UUID	FOREIGN KEY → users.id, NULL	Attending vet
cost	DECIMAL(10,2)	-	Treatment cost
notes	TEXT	-	Additional notes
next_appointment	DATE	-	Follow-up date
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `veterinarian_id` → `users(id)` ON DELETE SET NULL
- INDEX: `(animal_id, date)` - for timeline queries

Table: `vaccinations`

Django Model: `apps/livestock/models.py` → `Vaccination`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
animal_id	UUID	FOREIGN KEY → animals.id	Associated animal
vaccine_name	VARCHAR(100)	NOT NULL	Vaccine type
disease_target	VARCHAR(100)	-	Disease prevented
date_administered	DATE	NOT NULL	Vaccination date
next_due_date	DATE	-	Next dose due
batch_number	VARCHAR(50)	-	Vaccine batch
administered_by_id	UUID	FOREIGN KEY → users.id	Administering person
location	VARCHAR(100)	-	Injection site
completed	BOOLEAN	DEFAULT false	Vaccination series complete
notes	TEXT	-	Additional notes
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `administered_by_id` → `users(id)` ON DELETE SET NULL
- INDEX: `next_due_date` - for alert generation

Table: `weight_records`

Django Model: `apps/livestock/models.py` → `WeightRecord`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
animal_id	UUID	FOREIGN KEY → animals.id	Associated animal
date	DATE	NOT NULL	Measurement date
weight_kg	DECIMAL(6,2)	NOT NULL	Weight in kilograms
method	VARCHAR(50)	-	scale/estimation/visual
measured_by_id	UUID	FOREIGN KEY → users.id, NULL	Person who measured
notes	TEXT	-	Additional notes
created_at	TIMESTAMP	NOT NULL	Record creation

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `measured_by_id` → `users(id)` ON DELETE SET NULL
- INDEX: `(animal_id, date)` - for growth charts

Table: **`breeding_records`**

Django Model: `apps/livestock/models.py` → `BreedingRecord`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
dam_id	UUID	FOREIGN KEY → animals.id	Mother
sire_id	UUID	FOREIGN KEY → animals.id	Father
breeding_date	DATE	NOT NULL	Breeding/insemination date
method	VARCHAR(50)	NOT NULL	natural/artificial
pregnancy_status	VARCHAR(20)	-	confirmed/suspected/not_pregnant
pregnancy_check_date	DATE	-	Pregnancy verification date
expected_calving_date	DATE	-	Estimated birth date
actual_calving_date	DATE	-	Actual birth date
offspring_id	UUID	FOREIGN KEY → animals.id, NULL	Born calf
notes	TEXT	-	Additional notes
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `dam_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `sire_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `offspring_id` → `animals(id)` ON DELETE SET NULL
- INDEX: `expected_calving_date` - for calendar planning

📡 App 3: `apps/iot/` - Sensor & Device Data

Table: `devices`

Django Model: `apps/iot/models.py` → `Device`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
device_id	VARCHAR(50)	UNIQUE, NOT NULL, INDEXED	Hardware device ID (e.g., ESP32_001)
animal_id	UUID	FOREIGN KEY → animals.id, NULL	Assigned animal
firmware_version	VARCHAR(20)	DEFAULT '1.0.0'	Firmware version
battery_level	INTEGER	DEFAULT 100	Battery percentage (0-100)
status	VARCHAR(20)	DEFAULT 'active'	active/inactive/maintenance
last_seen	TIMESTAMP	-	Last communication time
created_at	TIMESTAMP	NOT NULL	Device registration
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- UNIQUE INDEX: `device_id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE SET NULL

Table: **`sensor_data`**

Django Model: `apps/iot/models.py` → `SensorData`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
device_id	UUID	FOREIGN KEY → devices.id	Source device
animal_id	UUID	FOREIGN KEY → animals.id, NULL	Associated animal (denormalized for speed)
timestamp	TIMESTAMP	NOT NULL, INDEXED	Reading timestamp
latitude	FLOAT	NOT NULL	GPS latitude
longitude	FLOAT	NOT NULL	GPS longitude
altitude	FLOAT	-	GPS altitude (meters)
satellites	INTEGER	DEFAULT 0	GPS satellite count
body_temperature	FLOAT	NOT NULL	Body temp (°C)
ambient_temperature	FLOAT	NOT NULL	Ambient temp (°C)
acc_x	FLOAT	NOT NULL	Accelerometer X (m/s^2)
acc_y	FLOAT	NOT NULL	Accelerometer Y (m/s^2)
acc_z	FLOAT	NOT NULL	Accelerometer Z (m/s^2)
activity_level	FLOAT	-	Calculated magnitude of acceleration
battery_level	INTEGER	DEFAULT 100	Device battery at time of reading
created_at	TIMESTAMP	NOT NULL	Record creation

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `device_id` → `devices(id)` ON DELETE CASCADE
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- COMPOSITE INDEX: `(device_id, timestamp DESC)` - for device timeline
- COMPOSITE INDEX: `(animal_id, timestamp DESC)` - for animal timeline
- INDEX: `timestamp` - for time-range queries

Notes:

- `activity_level` auto-calculated on save: `sqrt(acc_x² + acc_y² + acc_z²)`
- `animal_id` denormalized from device for faster queries

🤖 App 4: `apps/analytics/` - ML & Predictions

Table: `ml_models`

Django Model: `apps/analytics/models.py` → `MLModel`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Model name (e.g., health_v1)
version	VARCHAR(20)	NOT NULL	Model version
model_type	VARCHAR(50)	NOT NULL	health/behavior/estrus/anomaly
model_file	VARCHAR(200)	NOT NULL	Path to saved model file
accuracy	FLOAT	-	Model accuracy score
is_active	BOOLEAN	DEFAULT true	Currently in use
description	TEXT	-	Model description
created_at	TIMESTAMP	NOT NULL	Model creation
updated_at	TIMESTAMP	NOT NULL	Last update

Indexes:

- PRIMARY KEY: `id`
- UNIQUE: `name`

Table: `predictions`

Django Model: `apps/analytics/models.py` → `Prediction`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
animal_id	UUID	FOREIGN KEY → animals.id	Prediction subject
prediction_type	VARCHAR(50)	NOT NULL	health/behavior/estrus/anomaly
model_name	VARCHAR(100)	NOT NULL	Model used
model_version	VARCHAR(20)	NOT NULL	Model version
predicted_value	VARCHAR(100)	NOT NULL	Prediction result (e.g., "SICK", "GRAZING")
confidence_score	FLOAT	NOT NULL	Confidence (0.0-1.0)
input_data	JSON/TEXT	-	Input features used
metadata	JSON/TEXT	-	Additional context
predicted_at	TIMESTAMP	NOT NULL	Prediction timestamp

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- INDEX: `(animal_id, predicted_at DESC)` - for prediction history
- INDEX: `predicted_at` - for time-based queries

🚨 App 5: `apps/alerts/` - Notifications

Table: `alerts`

Django Model: `apps/alerts/models.py` → `Alert`

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
animal_id	UUID	FOREIGN KEY → animals.id, NULL	Related animal
alert_type	VARCHAR(50)	NOT NULL	health/vaccination/geofence/battery/weight/estrus
severity	VARCHAR(20)	NOT NULL	low/medium/high/critical
title	VARCHAR(200)	NOT NULL	Alert title
message	TEXT	NOT NULL	Alert description
is_read	BOOLEAN	DEFAULT false	User has seen alert
is_resolved	BOOLEAN	DEFAULT false	Issue resolved
resolved_at	TIMESTAMP	-	Resolution timestamp
resolved_by_id	UUID	FOREIGN KEY → users.id, NULL	User who resolved
created_at	TIMESTAMP	NOT NULL	Alert generation time

Indexes:

- PRIMARY KEY: `id`
- FOREIGN KEY: `animal_id` → `animals(id)` ON DELETE CASCADE
- FOREIGN KEY: `resolved_by_id` → `users(id)` ON DELETE SET NULL
- INDEX: `created_at DESC` - for recent alerts
- INDEX: `(is_resolved, severity)` - for active alerts dashboard

📋 Complete Django Models Code

File: `apps/core/models.py`

```python
```

```python
import uuid
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    ROLE_CHOICES = [
        ('admin', 'Administrator'),
        ('vet', 'Veterinarian'),
        ('worker', 'Ranch Worker'),
        ('viewer', 'Viewer'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='viewer')
    phone_number = models.CharField(max_length=20, blank=True)

    class Meta:
        db_table = 'users'

class Ranch(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=200)
    location = models.CharField(max_length=200)
    size_hectares = models.DecimalField(max_digits=10, decimal_places=2)
    owner = models.ForeignKey(User, on_delete=models.CASCADE, related_name='owned_ranches')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'ranches'
        verbose_name_plural = 'Ranches'

    def __str__(self):
        return self.name

class Animal(models.Model):
    GENDER_CHOICES = [('male', 'Male'), ('female', 'Female')]
    STATUS_CHOICES = [
        ('active', 'Active'),
        ('sick', 'Sick'),
        ('quarantine', 'Quarantine'),
        ('sold', 'Sold'),
        ('deceased', 'Deceased'),
```

```
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    tag_id = models.CharField(max_length=50, unique=True, db_index=True)
    name = models.CharField(max_length=100)
    breed = models.CharField(max_length=50)
    gender = models.CharField(max_length=10, choices=GENDER_CHOICES)
    birth_date = models.DateField()
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='active')
    ranch = models.ForeignKey(Ranch, on_delete=models.CASCADE, related_name='animals')
    sire = models.ForeignKey('self', on_delete=models.SET_NULL, null=True, blank=True, related_name='offspring_a
    dam = models.ForeignKey('self', on_delete=models.SET_NULL, null=True, blank=True, related_name='offspring_a
    photo = models.ImageField(upload_to='animals/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'animals'
        ordering = ['-created_at']

    def __str__(self):
        return f"{self.tag_id} - {self.name}"

    @property
    def age_months(self):
        from datetime import date
        today = date.today()
        return (today.year - self.birth_date.year) * 12 + today.month - self.birth_date.month
```

File: `apps/livestock/models.py`

```python
```

```python
import uuid
from django.db import models
from apps.core.models import Animal, User

class HealthRecord(models.Model):
    RECORD_TYPE_CHOICES = [
        ('checkup', 'Checkup'),
        ('treatment', 'Treatment'),
        ('injury', 'Injury'),
        ('illness', 'Illness'),
        ('surgery', 'Surgery'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='health_records')
    record_type = models.CharField(max_length=50, choices=RECORD_TYPE_CHOICES)
    date = models.DateField()
    diagnosis = models.TextField(blank=True)
    treatment = models.TextField(blank=True)
    medication = models.CharField(max_length=200, blank=True)
    dosage = models.CharField(max_length=100, blank=True)
    veterinarian = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name='hea
    cost = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)
    notes = models.TextField(blank=True)
    next_appointment = models.DateField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'health_records'
        ordering = ['-date']
        indexes = [
            models.Index(fields=['animal', '-date']),
        ]

    def __str__(self):
        return f"{self.animal.tag_id} - {self.record_type} - {self.date}"

class Vaccination(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='vaccinations')
    vaccine_name = models.CharField(max_length=100)
    disease_target = models.CharField(max_length=100, blank=True)
```

```
    date_administered = models.DateField()
    next_due_date = models.DateField(null=True, blank=True)
    batch_number = models.CharField(max_length=50, blank=True)
    administered_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name
    location = models.CharField(max_length=100, blank=True)
    completed = models.BooleanField(default=False)
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'vaccinations'
        ordering = ['-date_administered']
        indexes = [
            models.Index(fields=['next_due_date']),
        ]

    def __str__(self):
        return f"{self.animal.tag_id} - {self.vaccine_name} - {self.date_administered}"

class WeightRecord(models.Model):
    METHOD_CHOICES = [
        ('scale', 'Scale'),
        ('estimation', 'Estimation'),
        ('visual', 'Visual'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='weight_records')
    date = models.DateField()
    weight_kg = models.DecimalField(max_digits=6, decimal_places=2)
    method = models.CharField(max_length=50, choices=METHOD_CHOICES, blank=True)
    measured_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name='w
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'weight_records'
        ordering = ['-date']
        indexes = [
            models.Index(fields=['animal', '-date']),
        ]

    def __str__(self):
```

```
        return f"{self.animal.tag_id} - {self.weight_kg}kg - {self.date}"

class BreedingRecord(models.Model):
    METHOD_CHOICES = [
        ('natural', 'Natural'),
        ('artificial', 'Artificial Insemination'),
    ]
    PREGNANCY_CHOICES = [
        ('confirmed', 'Confirmed'),
        ('suspected', 'Suspected'),
        ('not_pregnant', 'Not Pregnant'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    dam = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='breeding_as_dam')
    sire = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='breeding_as_sire')
    breeding_date = models.DateField()
    method = models.CharField(max_length=50, choices=METHOD_CHOICES)
    pregnancy_status = models.CharField(max_length=20, choices=PREGNANCY_CHOICES, blank=True)
    pregnancy_check_date = models.DateField(null=True, blank=True)
    expected_calving_date = models.DateField(null=True, blank=True)
    actual_calving_date = models.DateField(null=True, blank=True)
    offspring = models.ForeignKey(Animal, on_delete=models.SET_NULL, null=True, blank=True, related_name='bir
    notes = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'breeding_records'
        ordering = ['-breeding_date']
        indexes = [
            models.Index(fields=['expected_calving_date']),
        ]

    def __str__(self):
        return f"{self.dam.tag_id} x {self.sire.tag_id} - {self.breeding_date}"
```

File: `apps/iot/models.py`

```
python
```

```python
import uuid
from django.db import models
from apps.core.models import Animal

class Device(models.Model):
    STATUS_CHOICES = [
        ('active', 'Active'),
        ('inactive', 'Inactive'),
        ('maintenance', 'Maintenance'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    device_id = models.CharField(max_length=50, unique=True, db_index=True)
    animal = models.ForeignKey(Animal, on_delete=models.SET_NULL, null=True, blank=True, related_name='devic
    firmware_version = models.CharField(max_length=20, default='1.0.0')
    battery_level = models.IntegerField(default=100)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='active')
    last_seen = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'devices'

    def __str__(self):
        return f"{self.device_id} ({self.status})"

class SensorData(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    device = models.ForeignKey(Device, on_delete=models.CASCADE, related_name='sensor_readings')
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='sensor_data', null=True, blank
    timestamp = models.DateTimeField(db_index=True)
    latitude = models.FloatField()
    longitude = models.FloatField()
    altitude = models.FloatField(null=True, blank=True)
    satellites = models.IntegerField(default=0)
    body_temperature = models.FloatField()
    ambient_temperature = models.FloatField()
    acc_x = models.FloatField()
    acc_y = models.FloatField()
    acc_z = models.FloatField()
    activity_level = models.FloatField(null=True, blank=True)
    battery_level = models.IntegerField(default=100)
```

```
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'sensor_data'
        ordering = ['-timestamp']
        indexes = [
            models.Index(fields=['device', '-timestamp']),
            models.Index(fields=['animal', '-timestamp']),
        ]

    def __str__(self):
        return f"{self.device.device_id} - {self.timestamp}"

    def save(self, *args, **kwargs):
        if self.acc_x is not None and self.acc_y is not None and self.acc_z is not None:
            import math
            self.activity_level = math.sqrt(self.acc_x**2 + self.acc_y**2 + self.acc_z**2)
        super().save(*args, **kwargs)
```

File: **apps/analytics/models.py**

```
python
```

```python
import uuid
from django.db import models
from apps.core.models import Animal

class MLModel(models.Model):
    MODEL_TYPE_CHOICES = [
        ('health', 'Health Prediction'),
        ('behavior', 'Behavior Classification'),
        ('estrus', 'Estrus Detection'),
        ('anomaly', 'Anomaly Detection'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100, unique=True)
    version = models.CharField(max_length=20)
    model_type = models.CharField(max_length=50, choices=MODEL_TYPE_CHOICES)
    model_file = models.CharField(max_length=200)
    accuracy = models.FloatField(null=True, blank=True)
    is_active = models.BooleanField(default=True)
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'ml_models'

    def __str__(self):
        return f"{self.name} v{self.version}"

class Prediction(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, related_name='predictions')
    prediction_type = models.CharField(max_length=50)
    model_name = models.CharField(max_length=100)
    model_version = models.CharField(max_length=20)
    predicted_value = models.CharField(max_length=100)
    confidence_score = models.FloatField()
    input_data = models.JSONField(null=True, blank=True)
    metadata = models.JSONField(null=True, blank=True)
    predicted_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'predictions'
```

```
        ordering = ['-predicted_at']
        indexes = [
            models.Index(fields=['animal', '-predicted_at']),
        ]

    def __str__(self):
        return f"{self.animal.tag_id} - {self.prediction_type}: {self.predicted_value}"
```

File: apps/alerts/models.py

```python
```

```python
import uuid
from django.db import models
from apps.core.models import Animal, User

class Alert(models.Model):
    ALERT_TYPE_CHOICES = [
        ('health', 'Health Warning'),
        ('vaccination', 'Vaccination Due'),
        ('geofence', 'Geofence Breach'),
        ('battery', 'Low Battery'),
        ('weight', 'Weight Concern'),
        ('estrus', 'Estrus Detected'),
    ]
    SEVERITY_CHOICES = [
        ('low', 'Low'),
        ('medium', 'Medium'),
        ('high', 'High'),
        ('critical', 'Critical'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE, null=True, blank=True, related_name='alerts'
    alert_type = models.CharField(max_length=50, choices=ALERT_TYPE_CHOICES)
    severity = models.CharField(max_length=20, choices=SEVERITY_CHOICES)
    title = models.CharField(max_length=200)
    message = models.TextField()
    is_read = models.BooleanField(default=False)
    is_resolved = models.BooleanField(default=False)
    resolved_at = models.DateTimeField(null=True, blank=True)
    resolved_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name='res
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'alerts'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['is_resolved', 'severity']),
        ]

    def __str__(self):
        return f"{self.severity.upper()}: {self.title}"
```

🔧 Database Setup Commands

bash

```
# Create all migrations
python manage.py makemigrations core
python manage.py makemigrations livestock
python manage.py makemigrations iot
python manage.py makemigrations analytics
python manage.py makemigrations alerts

# Apply migrations
python manage.py migrate

# Create superuser
python manage.py createsuperuser

# Seed database
python manage.py seed_data
```

📊 Entity Relationship Diagram (ERD)

✅ Summary Table

Table	Django App	Model Name	Primary Use
users	core	User	Authentication
ranches	core	Ranch	Ranch management
animals	core	Animal	Animal records
health_records	livestock	HealthRecord	Medical history
vaccinations	livestock	Vaccination	Vaccination tracking
weight_records	livestock	WeightRecord	Weight monitoring
breeding_records	livestock	BreedingRecord	Breeding management
devices	iot	Device	IoT collar registry
sensor_data	iot	SensorData	Sensor readings
ml_models	analytics	MLModel	ML model registry
predictions	analytics	Prediction	ML predictions
alerts	alerts	Alert	Notifications

Total: 12 tables across 5 Django apps